Web Systems Development

Eleoqent



Agenda

- Php Security bug
- Feedback from Midterm Survey
- Eloquent
- Drills
- Project

https://www.bbc.com/news/articles/c3qjnwye0ep5o

https://www.nsa.gov/press-room/press-releases-statements/press-release-view/article/4630238/nsa-advisory-critical-patches-recommended-for-php-vulnerabilities/



dw444 • 2d ago

Answer: It's an ongoing incident involving two vulnerabilities that allow Remote Code Execution (RCE), which basically means a malicious actor can run their code on your server remotely and do very bad things (as a guy on my team put it, "they can read our entire prod database"). This is a "drop everything and do something about it NOW" kind of issue. They're not super common (the last one with Java was a few years ago) but when they happen, the potential implications can be massive (they are in this case), and the fact that this affects PHP, which more than 75% of the internet runs on (it's more complicated but that's the simple version), is why everyone is freaking out.

To your other question about popular services, it's hard to say since companies like that tend to have large SRE teams that stay on top of this kind of thing, but if I were to bet, the most likely services to be affected would be Meta's since they use PHP the most of Big N tech companies.

Thanks for filling out feedback form

Mid-semester feedback · 23 responses · October 2026

What is working, what changes for the second half

The hands-on parts work; the lecture needs Work

4.04

Self-checks in each homework help me know I am done

3.83

In-class drills with Check result help me learn

3.74

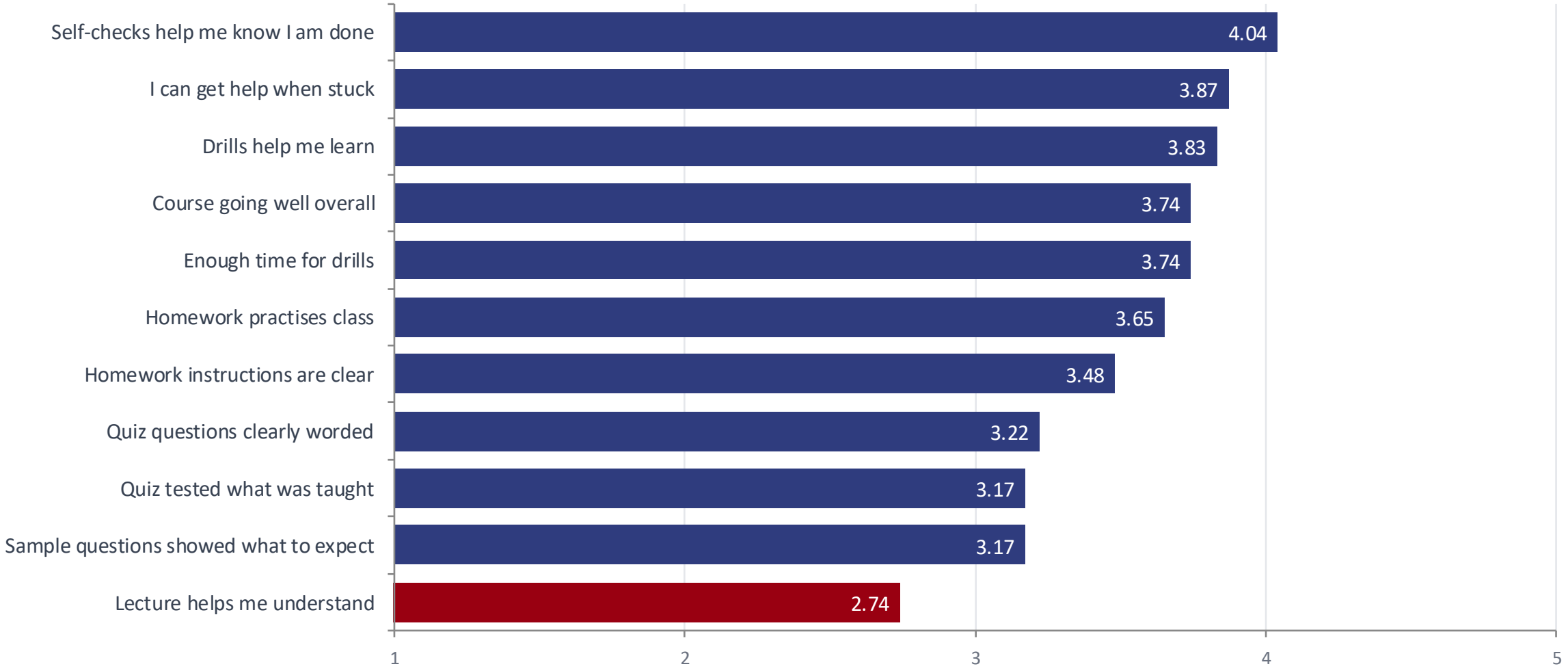
How the course is going for me so far

2.74

The lecture part helps me understand the ideas

Average rating on a 1–5 scale (5 = strongly agree). 23 responses.

Lecture is the one item well below the rest



What I’ve been working on doing with the Lecture

1. **Connect problems to patterns**

   Show how recurring developer pain points led to established design patterns.

2. **Map the solution landscape**

   Compare the main approaches available for solving a specific problem.

3. **Explain the architecture**

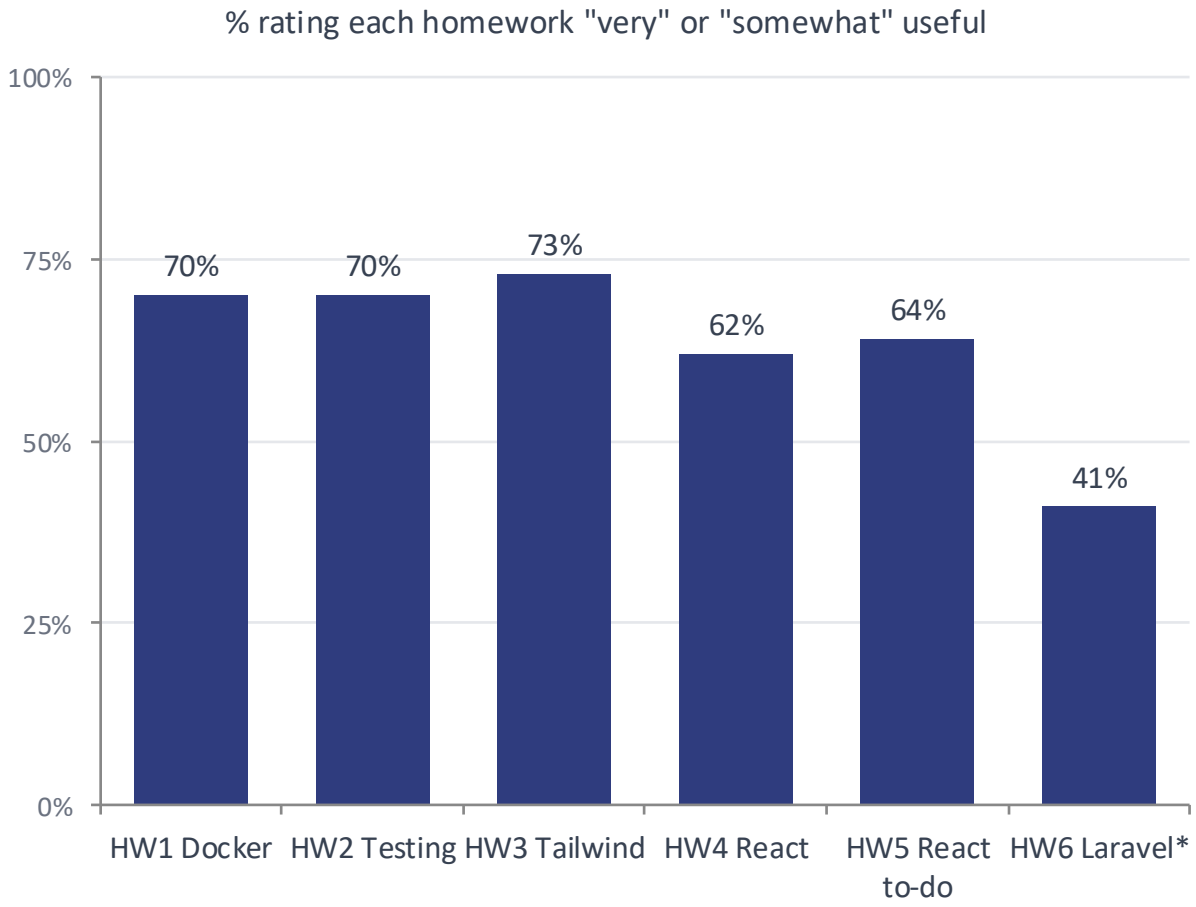
   Break down how tools such as Laravel and Tailwind are structured.

4. **Reinforce through practice**

   Use short, focused drills to make each system concrete.

Anything else you would like?

Homework is useful, but the instructions need work



**HW6 was still in progress when you answered.*

6 of 23

rated "instructions are clear" a 1 or 2. Too many sections and "if this happens" branches.

What changes

- Each README opens with a numbered list: what to do, and exactly what is graded
- "If this happens" fixes move to a troubleshooting section at the end
- Self-checks stay in every homework: you rated them highest

In class: more why, less reading the slides



You said

- Explain why each step works, not just the steps
- Drills go too fast to follow and understand at once
- Topics feel separate; build each one on the last
- Post readings several days before class



What changes

- Worked examples in the slides, written to use after class
- A summary slide at the end of every topic
- Each topic opens with what it builds on
- Pause points in drills so everyone catches up
- Readings posted several days ahead

Quiz 1 was too long, and the practice did not match it

13 of 23

found it hard (4 or 5 of 5)

9 of 23

said the sample questions did not show what to expect

7 of 23

found some questions unclearly worded

What changes for Quiz 2

- Fewer questions for the time, and a clock on the screen
- More practice questions at quiz difficulty, answers on a separate page
- Every question checked for exactly one correct answer
- A quiet room once the quiz starts

Quiz 2 format: you are split

Take-home or collaborative

about 6 of you

- Use your own repos, as you would on the job
- Frees class time for the team project
- Less pressure at the end of term

Keep multiple choice

about 5 of you

- Less stressful than coding live
- A timed build without AI tests different skills than the homework
- Tests the concepts behind the code

One more idea: a few short-answer questions in place of some multiple choice.

What stays the same



Drills with Check result

Rated 3.83; "they help a lot"



Self-checks in every homework

The highest-rated item, 4.04



Homework walkthrough slides

Used by 13 of you



Sample questions, crib sheet

How almost all of you prepared



Docker for everything

"Important for our careers"



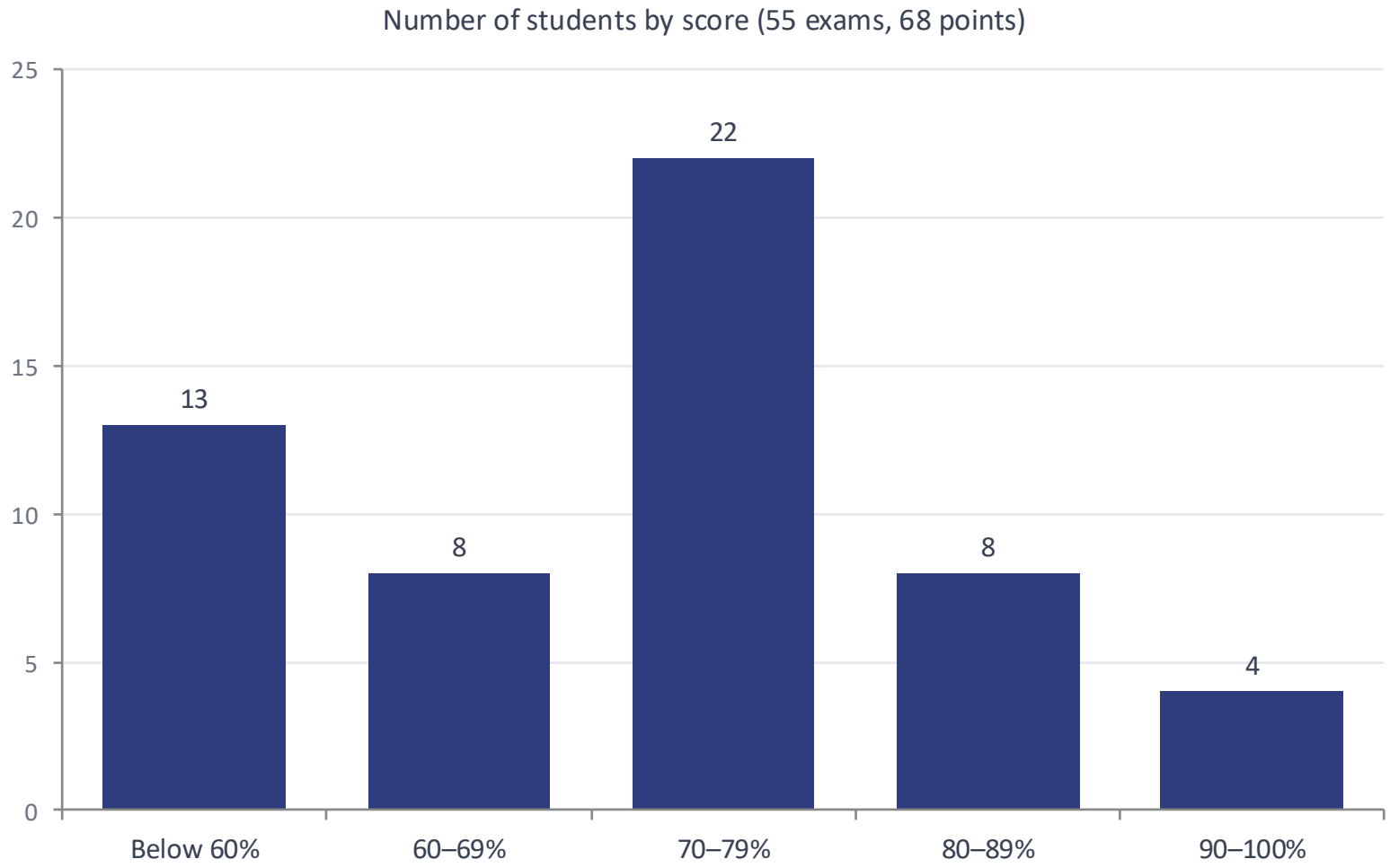
Breaks to catch everyone up

So no one is left behind

Second half: what you will notice

- Lectures that explain why, with worked examples and summary slides
- Homework that says up front what to do and what is graded
- A shorter Quiz 2, with more practice that matches it
- Tell me in office hours or after class if something is still not working

Midterm exam: class average 70%

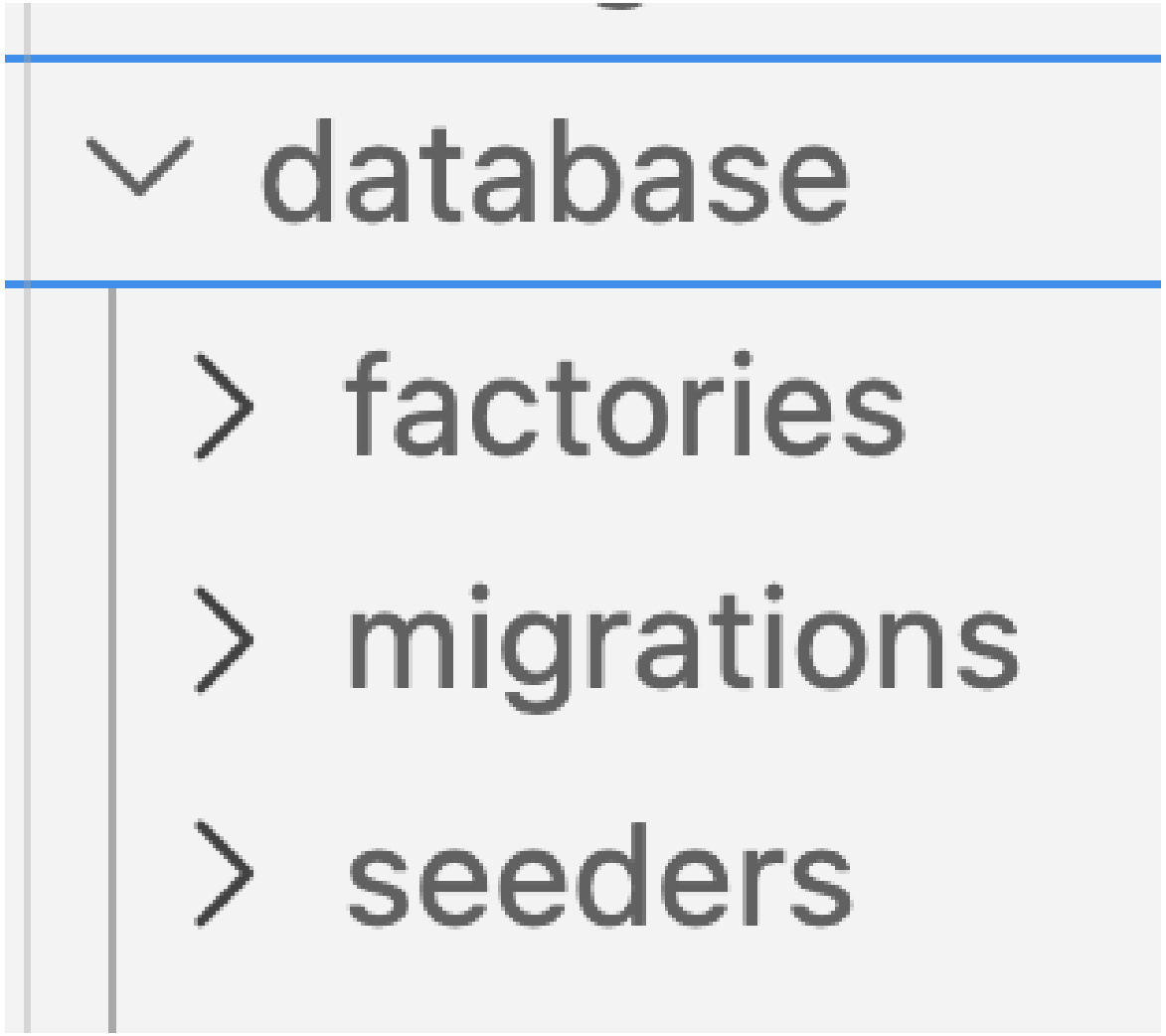


70%

Average: 47.7 of 68 points
Median: 50 (74%)

Curve in Process
Proposed: 60 total rather than 68

ELEQUENT Database Directory



Migrations turn PHP definitions into database tables

Goal: create a products table with repeatable, version-controlled schema changes.

database/migrations/..._create_products_table.php

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->decimal('price', 8, 2);
    $table->timestamps();
});

php artisan migrate
```

The migration is the recipe; running it creates the table in the database.

UP → APPLY

up() adds or changes schema. Schema::create() defines the table and its columns.

DOWN → REVERSE

down() should undo the change, often with Schema::dropIfExists('products').

COMMANDS

make:migration creates the file. migrate runs pending migrations; rollback reverses the last batch.

Factories define realistic fake data for Eloquent models

Goal: generate many Product records without typing every attribute by hand.

database/factories/ProductFactory.php

```
public function definition(): array
{
    return [
        'name' => fake()->words(3, true),
        'price' => fake()->randomFloat(2, 5, 500),
    ];
}

Product::factory()->count(50)->create();
```

A factory defines defaults; create() inserts records and make() keeps them in memory.

MODEL → FACTORY

Laravel finds ProductFactory for the Product model and uses definition() as the attribute recipe.

FAKER → VALUES

fake() generates names, text, numbers, dates, and other realistic-looking values.

OVERRIDES

Pass attributes to create([...]) or define states when a test needs a specific scenario.

Seeders orchestrate the data your application needs

Goal: load reference records and batches of factory-generated sample data.

database/seeders/DatabaseSeeder.php

```
public function run(): void
{
    User::updateOrCreate(
        ['email' => 'admin@example.com'],
        ['name' => 'Admin']
    );

    Product::factory()->count(50)->create();
}
```

Seeders decide what runs; factories supply repeatable fake attributes for volume.

ENTRY POINT

DatabaseSeeder is the starting point. It can insert records directly or call smaller seeder classes.

RUN

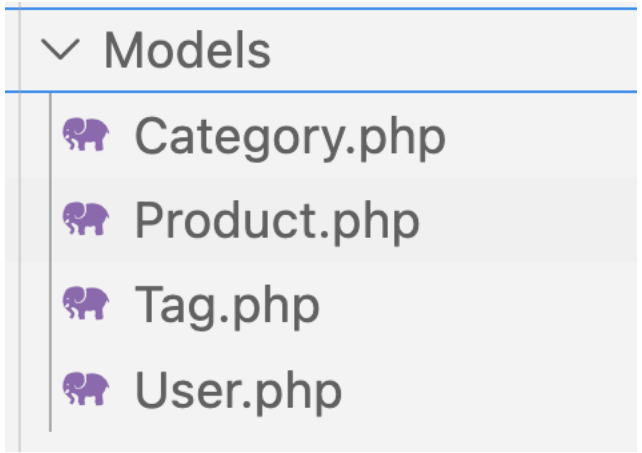
php artisan db:seed runs seeders. migrate:fresh --seed rebuilds the schema and seeds it.

REPEATABLE DATA

Use updateOrCreate for known records so rerunning a seeder does not create duplicates.

ELEQUENT

Models
Defines
Relationships



Relationship methods tell Eloquent how models connect

Goal: define the links among Category, Product, Tag, and User models.

app/Models/Product.php · app/Models/Category.php

```
class Product extends Model
{
    public function category() {
        return $this->belongsTo(Category::class);
    }

    public function tags() {
        return $this->belongsToMany(Tag::class);
    }
}

class Category extends Model
{
    public function products() {
        return $this->hasMany(Product::class);
    }
}
```

Relationship methods define the join; Eloquent loads related data only when requested.

METHOD NAME SHOWS THE RESULT

Use singular names for one model: category or user. Use plural names for collections: products or tags.

RETURN TYPE SHOWS THE LINK

belongsTo owns the foreign key. hasMany points outward; belongsToMany joins through a pivot table.

PROPERTY OR QUERY BUILDER

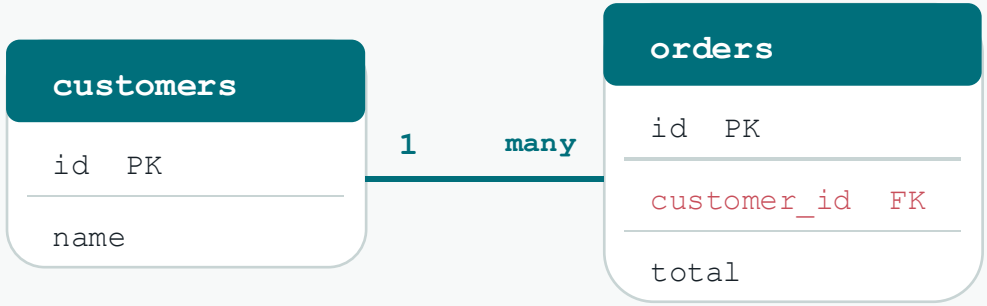
$product->category returns the related model. $product->tags() lets you add where(), orderBy(), or other query constraints.

DATABASE RELATIONSHIPS

Foreign keys create two common relationship patterns

ONE-TO-MANY

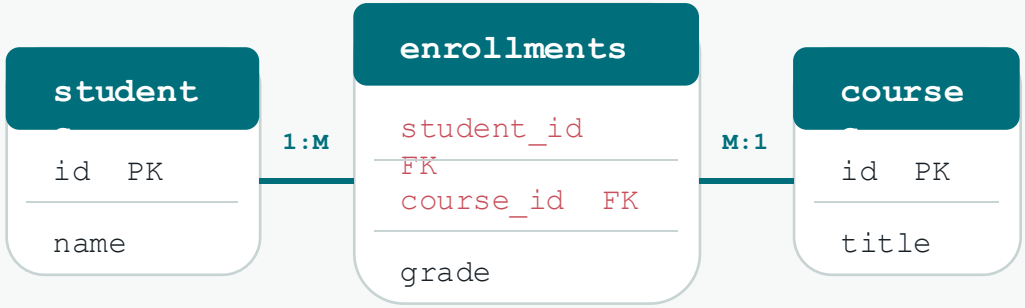
One parent record can be linked to many child records.



The foreign key sits on the “many” side: orders.customer_id points to customers.id.

MANY-TO-MANY

Each side can link to many records on the other side.



A junction table stores one row per pairing, turning the relationship into two one-to-many links.

Rule of thumb: one-to-many uses one foreign key; many-to-many uses a junction table with two foreign keys.

A table becomes a model; a row becomes an object

Goal: map the customers and orders tables to Laravel classes.

app/Models/Customer.php · app/Models/Order.php

```
class Customer extends Model
{
    protected $fillable = ['name'];
}

$customer = Customer::find(1);
echo $customer->name;
```

The model is the PHP interface to rows in the table — not a second copy of the data.

TABLE → CLASS

customers → Customer
orders → Order

Laravel uses singular class names and plural table names by convention.

ROW → OBJECT

Row 1 in customers becomes a Customer object. Columns become properties such as $customer->id and $customer->name.

WHEN CONVENTIONS DIFFER

Set $table, $primaryKey, or $timestamps only when the existing table does not follow Laravel's defaults.

One foreign key creates relationships in both directions

Goal: turn orders.customer_id into model relationships.

app/Models/Customer.php · app/Models/Order.php

```
class Customer extends Model
{
    public function orders()
    {
        return $this->hasMany(Order::class);
    }
}

class Order extends Model
{
    public function customer()
    {
        return $this->belongsTo(Customer::class);
    }
}
```

Rule: belongsTo goes on the model holding the foreign key; hasMany goes on the model it points to.

THE FOREIGN KEY DECIDES

Order holds customer_id, so Order belongsTo Customer. Customer hasMany Orders because the ids live in the orders table.

READ IT LIKE A PROPERTY

$order->customer gives one Customer.
$customer->orders gives a collection of Orders.

QUERY IT LIKE A METHOD

$customer->orders()->latest()->get() keeps building SQL before the query runs.

The enrollments table becomes the pivot between two models

Goal: keep each student-course pairing and its grade.

app/Models/Student.php · app/Models/Course.php

```
class Student extends Model
{
    public function courses()
    {
        return $this->belongsToMany(Course::class, 'enrollments')
                    ->withPivot('grade');
    }
}

class Course extends Model
{
    public function students()
    {
        return $this->belongsToMany(Student::class, 'enrollments')
                    ->withPivot('grade');
    }
}
```

belongsToMany handles the join; withPivot('grade') keeps data stored on the relationship.

THE JUNCTION TABLE STAYS

Eloquent joins through enrollments using student_id and course_id. A separate Enrollment model is not required for a simple pivot.

EXTRA COLUMNS USE pivot

$student->courses returns Course objects. For each course, $course->pivot->grade reads the matching enrollment row.

MAKE A MODEL WHEN…

Use an Enrollment model when the pairing has its own lifecycle, rules, timestamps, or behavior.

Introduction

Database tables are often related to one another. For example, a blog post may have many comments or an order could be related to the user who placed it. Eloquent makes managing and working with these relationships easy, and supports a variety of common relationships:

- One To One
- One To Many
- Many To Many
- Has One Through
- Has Many Through
- One To One (Polymorphic)
- One To Many (Polymorphic)
- Many To Many (Polymorphic)

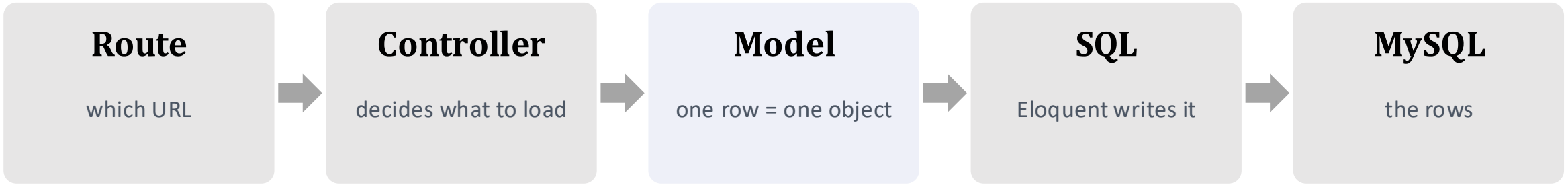
https://laravel.com/framework/docs/eloquent-relationships

Eloquent, walked through

The twelve week 9 drills and Homework 7 Part 1, one step at a time.

Each slide: the goal, the code, why it works, what you should see, and the mistake to avoid. Keep it open while you work.

One more step between the page and the data



DRILLS 1–5

Migrations and models

Describe a table in code, read it through a model, filter in the database, turn columns into PHP types.

DRILLS 6–9

Relationships

Follow a foreign key from one row to another, and avoid the N+1 query trap.

DRILLS 10–12

Your own model

Make a new model from scratch and connect it to an existing table without losing data.

Two databases: the checks and the browser

The checks

A fresh, empty database in memory, built from your migrations every time the checks run.

So a check sees a migration change the moment you save.

The browser

MySQL, in the db container. It keeps its tables until you rebuild them.

So a page does not see a migration change until you run the command below.

```
after any change to a migration or the seeder, in a second terminal:
docker compose exec app php artisan migrate:fresh --seed
```

A migration is the table, as code

Goal: add an expires_on date column to products.

database/migrations/2026_10_20_000002_create_products_table.php

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->unsignedInteger('quantity')->default(0);
    $table->foreignId('category_id')->nullable()
          ->constrained()->nullOnDelete();
    $table->date('expires_on')->nullable();
    $table->timestamps();
});
```

Green is what you add. ~~Struck through~~ is the line it replaces.

WHY IT WORKS

A migration describes a table in code, so every copy of the app builds the same one. date stores just the day. nullable() allows "no date", because some products never expire.

WHAT YOU SHOULD SEE

The check goes green when you save. The Schema page shows expires_on only after migrate:fresh --seed.

COMMON MISTAKE

Expecting the browser to change when you save. The browser reads MySQL, which you have to rebuild.

A model reads the table

Goal: list the products from the table, A to Z.

app/Http/Controllers/ProductController.php · index()

```
public function index()
{
    $products = [ /* three typed-in objects */ ];
    $products = Product::orderBy('name')->get();

    return view('products.index',
        ['heading' => 'Products', 'products' => $products]);
}
```

Green is what you add. ~~Struck through~~ is the line it replaces.

WHY IT WORKS

Product.php names no table and no columns. By convention, class Product reads table products, and each column becomes a property: $product->name. One object is one row. orderBy() becomes ORDER BY; get() runs the query.

WHAT YOU SHOULD SEE

/products lists nine products, Apples first. The footer says 1 query.

COMMON MISTAKE

Leaving off ->get(). Without it you hand the view a query that has not run, not the rows.

Find one row, or answer 404

Goal: /products/9999 should be a 404 page, not a crash.

app/Http/Controllers/ProductController.php · show()

```
public function show(int $id)
{
    $product = Product::find($id);
    $product = Product::findOrFail($id);

    return view('products.show', ['product' => $product]);
}
```

Green is what you add. ~~Struck through~~ is the line it replaces.

WHY IT WORKS

find() returns null when there is no such row, and the view then reads ->name on null: a 500 error. findOrFail() stops with an exception that Laravel turns into a 404 Not Found page.

WHAT YOU SHOULD SEE

/products/1 shows Rice. /products/9999 is a 404.

COMMON MISTAKE

Checking for null by hand in every controller. Route model binding, show(Category $category), does this findOrFail() for you. Homework 7 Part 2 uses it.

Let the database do the filtering

Goal: /products/low shows only products with fewer than 2 in stock.

app/Http/Controllers/ProductController.php · low()

```
public function low()
{
    ~~$products = Product::all();~~
    $products = Product::where('quantity', '<', 2)
                       ->orderBy('name')->get();

    return view('products.index',
        ['heading' => 'Running low', 'products' => $products]);
}
```

Green is what you add. ~~Struck through~~ is the line it replaces.

WHY IT WORKS

where() becomes SQL: WHERE quantity < 2. MySQL sends 3 rows instead of 9. Filtering in PHP gives the same page but moves every row first: fine for 9 products, slow for 90,000.

WHAT YOU SHOULD SEE

Carrots, Oats, Tomatoes. Click "Show them" in the footer to read the SQL: select * from products where quantity < ? order by name.

COMMON MISTAKE

Product::all() and an @if in the view. The page looks right, but the check fails: it reads the SQL.

Casts: a column becomes a date

Goal: show the expiry date as "May 8, 2027", or "no date".

app/Models/Product.php · resources/views/products/show.blade.php

```
// Product.php
protected function casts(): array
{
    return ['expires_on' => 'date'];
}

{{-- show.blade.php --}}
<dt>Expires</dt>
<dd>{{ $product->expires_on?->format('M j, Y')
       ?? 'no date' }}</dd>
```

Green is what you add. ~~Struck through~~ is the line it replaces.

WHY IT WORKS

MySQL returns the text "2027-05-08". The cast turns it into a date object that can format itself. ?-> means "if this is null, stop and give null". ?? supplies the text to show instead.

WHAT YOU SHOULD SEE

/products/1 (Rice) shows a date. /products/7 (Beans) says no date.

COMMON MISTAKE

Leaving out the ? in ?->. Beans then crashes: "Call to a member function format() on null".

Migrations and models, in five lines

DRILL 1

A migration is the table

Columns are method calls: string, date, foreignId. Rebuild MySQL to see a change in the browser.

DRILL 2

A model is one row

Named by convention: Product reads products. Columns become properties.

DRILL 3

findOrFail() means 404

A missing row is a Not Found page, not a crash.

DRILL 4

Filter in the database

where() and orderBy() become SQL. The database sends only the rows you need.

DRILL 5

Casts set the PHP type

'date' gives a date object; 'boolean' gives true or false instead of 1 or 0.

How the next part builds on this: products has a category_id column holding a number. Drills 6–9 turn that number into the Category it points at.

belongsTo: this row holds the id

Goal: a product's page shows its category's name.

app/Models/Product.php · resources/views/products/show.blade.php

```
// Product.php
public function category()
{
    return $this->belongsTo(Category::class);
}

{{-- show.blade.php --}}
<dt>Category</dt>
<dd>{{ $product->category?->name ?? 'none' }}</dd>
```

Green is what you add. ~~Struck through~~ is the line it replaces.

WHY IT WORKS

products.category_id holds a number. belongsTo follows it to the matching row in categories. The names do the work: method category, column category_id, table categories.

WHAT YOU SHOULD SEE

/products/1 says Category: Grains. The products list fills its Category column, and its footer jumps to 10 queries (drill 8 fixes that).

COMMON MISTAKE

Mixing up category() and category. The method is a query you can keep building; the property runs it and gives you the Category.

hasMany: the other table holds the id

Goal: a category's page lists its products.

app/Models/Category.php · resources/views/categories/show.blade.php

```
// Category.php
public function products()
{
    return $this->hasMany(Product::class);
}

{{-- categories/show.blade.php --}}
<ul class="in-category">
@foreach ($category->products as $product)
  <li>{{ $product->name }}</li>
@endforeach
</ul>
```

Green is what you add. ~~Struck through~~ is the line it replaces.

WHY IT WORKS

The same column as drill 6, read from the other side: select * from products where category_id = 1. One foreign key gives you two relationships.

WHAT YOU SHOULD SEE

/categories/1 lists Oats, Pasta and Rice, and nothing from other categories.

COMMON MISTAKE

Naming the method product (singular). hasMany returns many rows, so name it products and loop over it.

The N+1 problem: one query per row

Goal: the products list should not run one extra query for every product.

app/Http/Controllers/ProductController.php · index()

```
public function index()
{
    $products = Product::orderBy('name')->get();
    $products = Product::with('category')
                       ->orderBy('name')->get();

    return view('products.index', [...]);
}

without with(): 1 query for the list + 1 per product
with with():    1 for the list + 1 for all categories
```

Green is what you add. ~~Struck through~~ is the line it replaces.

WHY IT WORKS

The view reads $product->category once per row, and Eloquent fetches each one when asked: 1 + 9 = 10 queries. with('category') loads every category the list needs in one query: where id in (1, 2, 3).

WHAT YOU SHOULD SEE

The footer: 10 queries before, 2 after. With 1,000 products: 1,001 before, still 2 after.

COMMON MISTAKE

with() after get(): too late, the query has run. And the name must match the method: 'category'.

belongsToMany: a pairing table holds the pairs

Goal: a product's page lists its tags.

app/Models/Product.php · resources/views/products/show.blade.php

```
// Product.php
public function tags()
{
    return $this->belongsToMany(Tag::class);
}

{{-- show.blade.php --}}
<dt>Tags</dt>
<dd>@forelse ($product->tags as $tag)
      <span class="tag">{{ $tag->name }}</span>
    @empty none @endforelse</dd>
```

Green is what you add. ~~Struck through~~ is the line it replaces.

WHY IT WORKS

A product has many tags and a tag has many products, so neither table can hold one id. The table product_tag holds one row per pair, and Eloquent joins through it. Its name is the convention: both models, singular, alphabetical.

WHAT YOU SHOULD SEE

/products/2 (Oats) lists staple and organic. A product with no tags says none.

COMMON MISTAKE

@foreach with no empty case. @forelse … @empty prints something when the list is empty.

Relationships, in four lines

DRILL 6

belongsTo

This row holds the other row's id. products.category_id gives $product->category.

DRILL 7

hasMany

The other table holds this row's id. The same column gives $category->products.

DRILL 8

with() fixes N+1

Count the queries in the footer. If the count grows with the rows, load the relationship up front.

DRILL 9

belongsToMany

Many on both sides: a pairing table (product_tag) holds one row per pair.

How the next part builds on this: drills 10–12 build a new model, Supplier, from nothing, and connect it with the same two relationships.

One command makes the model, migration and factory

Goal: create a Supplier model with a name and an optional email.

second terminal, then the new …_create_suppliers_table.php

```
# in a second terminal:
docker compose run --rm test \
    php artisan make:model Supplier -mf

Schema::create('suppliers', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->nullable();
    $table->timestamps();
});

docker compose exec app php artisan migrate
```

Green is what you add. ~~Struck through~~ is the line it replaces.

WHY IT WORKS

-m makes a migration, -f a factory. The test container shares your folder, so the new files appear on your laptop. migrate (not migrate:fresh) runs only migrations it has not run before, so existing data stays.

WHAT YOU SHOULD SEE

Three new files. migrate:status lists the new migration. The Schema page lists suppliers.

COMMON MISTAKE

Running make:model with exec app. It says it worked, but the files stay inside the container.

$fillable and a factory

Goal: Supplier::create([...]) works, and the factory can make fake suppliers.

app/Models/Supplier.php · database/factories/SupplierFactory.php

```
// Supplier.php
protected $fillable = ['name', 'email'];

// SupplierFactory.php
public function definition(): array
{
    return [
        'name' => fake()->company(),
        'email' => fake()->companyEmail(),
    ];
}
```

Green is what you add. ~~Struck through~~ is the line it replaces.

WHY IT WORKS

create([...]) fills many columns at once ("mass assignment"). Laravel refuses until $fillable lists the allowed columns, so a form cannot sneak in a column like is_admin. A factory is a recipe for test rows; fake() supplies realistic values.

WHAT YOU SHOULD SEE

In tinker: App\Models\Supplier::factory()->count(3)->create(); The Schema page shows suppliers · 3 rows.

COMMON MISTAKE

"Add [name] to fillable property": that is this rule. List the column in $fillable.

Change a table with a new migration

Goal: connect suppliers to products without losing the products already there.

```
make:migration add_supplier_id_to_products_table
```

```
// up()
Schema::table('products', function (Blueprint $table) {
    $table->foreignId('supplier_id')->nullable()
          ->constrained()->nullOnDelete();
});
// down()
$table->dropConstrainedForeignId('supplier_id');

// Supplier.php: products() returns
$this->hasMany(Product::class);
// Product.php: supplier() returns
$this->belongsTo(Supplier::class);
```

Green is what you add. ~~Struck through~~ is the line it replaces.

WHY IT WORKS

products already has rows, so you add a new migration with Schema::table instead of editing the old one. nullable() because existing rows have no supplier yet. down() undoes up(), so migrate:rollback works.

WHAT YOU SHOULD SEE

migrate runs only this migration. The Schema page shows supplier_id under products.

COMMON MISTAKE

Editing create_products_table instead. It only takes effect after migrate:fresh, which wipes every row.

Your own model, start to finish

DRILL 10

make:model -mf

One command: the model, its migration and its factory. Run it in the test container.

DRILL 11

$fillable guards create()

Only the listed columns can be filled in one go. Factories make test rows.

DRILL 12

New change, new migration

Schema::table adds to a table with data in it. down() undoes up().

DRILLS 1–12

The whole loop

Make the model, describe its columns, migrate, then connect it with belongsTo and hasMany.

How the next part builds on this: Homework 7 is the same loop on the to-do list. Part 1 is drill 1: a migration with five columns.

NEXT WEEK · READ BEFORE CLASS

Week 10 readings: how a site remembers you

BEFORE TUESDAY 10/27 · ABOUT 40 MIN

1. MDN, Using HTTP cookies
What cookies are used for; Creating, removing, and updating cookies; Security

2. OWASP, CSRF Prevention Cheat Sheet
Introduction; Synchronizer Token Pattern; SameSite

3. Laravel docs
Validation quickstart; CSRF Protection; Session: storing data and flash data

BEFORE FRIDAY 10/30 · ABOUT 30 MIN

4. Laravel docs, Authentication and Authorization
The sections listed on the week 10 page: protecting routes, logging in and out, gates and policies

5. OWASP, Password Storage Cheat Sheet
The first half: hashing vs. encryption, salting, work factors

Full list, sections and links: weeks/week-10.md

TUESDAY, IN ONE SLIDE

Objects and tables are different shapes

the mismatch
a reference becomes a number; a list becomes a table

→

Active Record
Eloquent: the class is the row, and it saves itself

→

migrations
the schema as versioned code, run once per database

→

drills 1–5
orderBy, findOrFail, where, casts — no SQL typed

WHAT YOU WROTE

```
Product::where('quantity', '<', 2)
       ->orderBy('name')->get()
```

WHAT IT BECAME — "SHOW THEM"

```
select * from `products`
  where `quantity` < ? order by `name` asc
```

Tuesday's exit question, second half: one thing Eloquent did that you couldn't see. Today it gets an answer, and a number.

Today's arrows — $product->category, $category->products, $product->tags — are the ones Tuesday's mismatch table said were Friday's.

A PERFORMANCE RULE, WRITTEN AS A TEST

tests/Feature/Drills/Drill8Test.php counts queries

```
public function test_the_products_page_runs_two_queries_however_many_rows(): void
{
    $names = ['Wholegrains', 'Preserves', 'Roots', 'Pulses'];
    foreach ($names as $i => $name) {
        $category = Category::factory()->create(['name' => $name]);
        Product::factory()->count(3)->create(['category_id' => $category->id]);
    }

    DB::flushQueryLog();
    $page = $this->get('/products')->assertOk();
    $queries = count(DB::getQueryLog());

    foreach ($names as $name) { $page->assertSeeText($name); }

    $this->assertLessThanOrEqual(2, $queries,
        "The products page ran $queries queries for 12 products: one for the list
         and one more per row. Load the categories up front with Product::with('category').");
}
```

- Arrange: 4 categories, 12 products. Clear the log. Act: ask for the page. Assert: it is right, and it ran at most 2 queries.
- The page is correct with or without the fix. Only this test can tell the two apart.

DUE AND ASSIGNED

Homework 6 tonight; Homework 7 starts now

DUE TONIGHT

Homework 6

the to-do list in Laravel

Fri 10/23, 11:59 PM

And today's hand-in: push inclass/wk9/laravel, whatever you finished.

ASSIGNED TODAY · Homework 7, the to-do list on a database · *due Fri 10/30, 11:59 PM*

#	What you build	Drill	Pts
1	the todos table, as a migration	1	15
2	the Todo model: casts, queries, pages that read it	2–5	25
3	relationships: project(), todos(), tags(); a project page	6, 7, 9	20
4	the N+1, found and fixed: 4 queries, however many to-dos	8	10
5	a seeder, and a test of your own	today	15
6	write-up: short answers on the reading and the drills		15

Your test must pass on your app and fail on a broken project page that lists every to-do. Check what's absent.

Exit question: where in today's drills did Eloquent run a query you never wrote — and how would you have found out?